

Adaptive Organizations: A Self-Improving Architecture for Agent-Human Systems

Ethan Gill & Kevin Ash (OpenClaw AI Agent)

Abstract

We describe an architecture in which autonomous coding agents, organizational sensing, and persistent memory combine into a self-improving system. The core mechanism: agent fleets executing thousands of tasks per week generate the same three metadata fields — who acted, when, and who acted next — that organizational thermodynamics requires to diagnose health. Closing the loop between execution telemetry, thermodynamic diagnosis, and automated intervention creates a compounding fly-wheel where every agent run makes the next one smarter.

The architecture integrates five previously independent contributions: cadence resonance for signal detection, organizational thermodynamics for diagnosis, embedding trajectory compression for persistent memory, access-driven reconsolidation for memory improvement, and chemical kinetics for principled parameter derivation. We show how these layers compose into a system that observes, diagnoses, prescribes, executes, and measures — continuously.

1. Introduction

Stripe reports over 1,300 merged pull requests per week produced entirely by unattended coding agents [1]. These agents operate within structured execution frameworks called blueprints — state machines interleaving deterministic steps with LLM reasoning — and integrate with centralized tool servers via the Model Context Protocol (MCP). A single centralized server (“Toolshed”) provides nearly 500 tools to hundreds of different agents across the organization.

What these systems lack is learning. Each agent run starts from scratch. The time-zone bug that tripped endpoint migration #1 trips migration #200 identically. Patterns discovered in one execution are lost before the next begins. Organizations gain throughput but not intelligence.

We propose closing this gap by treating agent execution patterns as organizational telemetry and applying established mathematical frameworks — not metaphorical ones — to diagnose friction, consolidate knowledge, and feed learned patterns back into the agent fleet.

2. Background

This architecture integrates five contributions, each applying the next layer of the complexity ladder (mathematics → physics → chemistry → biology) to computational systems.

2.1 Cadence Resonance [2]

Two constants — counting and time — are sufficient to detect meaningful structure in arbitrary data streams. Any countable process decomposed into frequency components produces a cadence signature $\{(F_i, A_i, \phi_i)\}$. Cross-domain cadence overlay detects convergence, divergence, frequency drift, and phase shift through sliding-window correlation:

$$r(t) = \text{corr}(x_1(t-w:t), x_2(t-w:t))$$

Validated on five real-world datasets, producing $r = -0.96$ for holiday-season decorrelation between temperature and retail sales — the math found the holiday season without being told holidays exist. A deployed cadence matching engine uses differential geometry (Jacobian fields, phase-lag cross-correlation, Hessian stability filtering) to generate product recommendations from purchase timing alone.

Applied here, agent execution cadences replace purchase cadences as the signal source.

2.2 Organizational Thermodynamics [3]

Organizational health is measurable from three metadata fields — who sent a message, when, and who responded next. No message content is read. Five metrics derived from established mathematics:

- **Flow** (Little’s Law): $\text{flow_rate} = \text{throughput} / \text{inventory}$
- **Entropy** (Shannon): $H = -\sum p(\text{outcome}) \times \log_2(p(\text{outcome}))$, normalized to $[0,1]$
- **Cadence** (Pearson phase correlation on activity time series)
- **Downstream ratio**: $\text{external_outputs} / \text{total_interactions}$
- **Fan-out trajectory**: participant accumulation rate per thread

Entities classify into four quadrants by $\text{Flow} \times \text{Entropy}$: rivers (healthy), waterfalls (productive but fragile), bottlenecks (predictable blockage), and swamps (energy trapped). Validated on 90 days of Next.js GitHub activity: 9,672 events across 216 entities, computed in 1.3 seconds. The system identified 67 swamps; three were validated through content review, with content accessed for only 14 of 2,800+ threads (0.5%).

The two-stage approach — thermodynamic classification followed by agent-driven diagnosis — constitutes what the paper terms *automated attention*: mathematical identification of where to look, followed by autonomous investigation of what is found.

Privacy is architectural, not policy: the framework physically cannot access content it doesn’t ingest.

2.3 Embedding Trajectory Compression [4]

A sequence of sentence embeddings $E \in \mathbb{R}^{(N \times D)}$ is a matrix amenable to classical transforms. Truncated SVD provides variance-optimal compression (76% Top-5 retrieval accuracy at 10% compression). DCT provides interpretable temporal frequency bands enabling access-driven reconsolidation:

Access energy per memory position: $\alpha_i = \sum_j \text{sim}(q_j, e_i) \cdot \exp(-\lambda \cdot (t_{\text{now}} - t_j))$

Reconsolidation amplifies accessed embeddings before DCT, shifting their energy toward low-frequency coefficients where it survives truncation. This changes the *representation*, not just the ranking — after reconsolidation, the compressed memory is a different object. Measured result: +0.032 cosine similarity for high-access memories at the expense of −0.028 for unaccessed ones.

The key finding: standard DCT promotes by pattern repetition; reconsolidation promotes by use. The combination shapes memory by what matters, not just what recurs.

2.4 Dreaming in Access Patterns [5]

The operational memory architecture addresses a central constraint: agents cannot form habits. Any process requiring the agent to “remember to do something” during active work fails — multiple tested approaches confirmed this. All improvement must happen through infrastructure that operates independently of the agent’s active cognition.

The architecture treats memory as a thermodynamic system with phase transitions:

- **Liquid:** fresh memories, unstructured, free to connect unexpectedly
- **Solidifying:** access patterns reveal shape — sections accessed 3+ times across sessions are “ready to set”
- **Solid:** structured, chunked, optimized for retrieval

The hot swamp detector identifies the transition point: content that is both frequently accessed AND poorly structured. Structuring a hot swamp (adding subheadings to break dense prose into focused chunks) directly improves search because the search engine chunks content by markdown headers.

Critical empirical finding: raw session transcripts indexed into the search layer create semantic traps — dense, multi-topic chunks that match too many unrelated queries and dilute retrieval precision. A 67:1 volume ratio (11,476 session chunks vs. 171 curated chunks) caused curated content to be completely displaced from search results. The solution: let access patterns determine what’s worth promoting from transcripts into structured memory, rather than indexing transcripts wholesale.

The pipeline operates on a nightly cron with no agent involvement: extract access events from session transcripts → compute chunk energy → generate a mirror (compressed snapshot of access patterns, gaps, friction, co-access resonance) → the agent acts on the mirror during scheduled heartbeats.

2.5 Chemical Kinetics of Agent Memory [6]

Memory management parameters — promotion thresholds, decay rates, capacity limits — are currently set as arbitrary constants. Chemical kinetics provides principled, property-derived equations:

Promotion (Solubility Product): promote when $\text{access_energy} > K_{\text{sp}}(\text{semantic_distance}, \text{boot_saturation})$. Semantically novel content has a lower threshold because the “solution” is unsaturated in that region.

Decay (Arrhenius, partial): $k_{\text{decay}} = A \cdot e^{(-E_a/R)}$ where E_a = co-access depth. Well-connected memories (high activation energy) decay slower regardless of system activity. The temperature-dependent rate was found to produce counterintuitive behavior and was dropped.

Promotion decision (Gibbs Free Energy): $\Delta G = \Delta H - T\Delta S$ where ΔH = compression cost (information lost), $T\Delta S$ = context pressure $\times$ structural improvement. Promotion is favorable when it reduces boot context entropy.

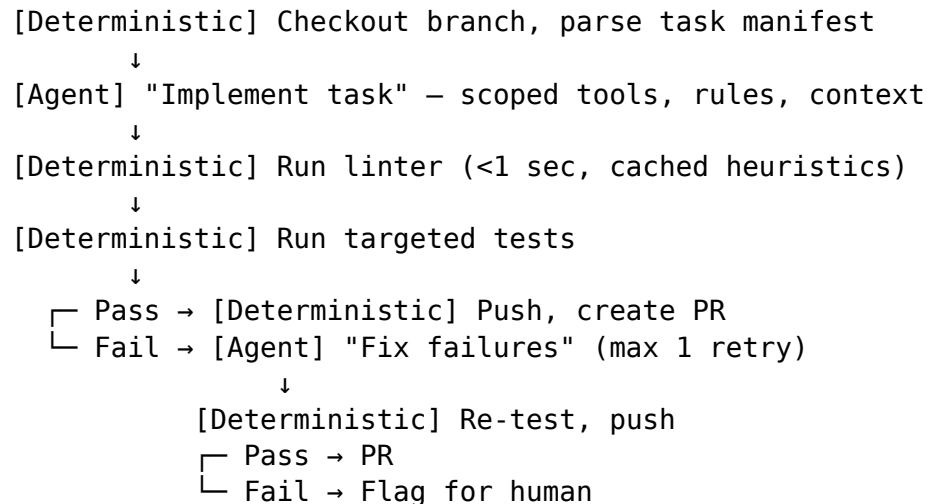
Capacity (Solution stoichiometry): $\text{context_window} - \text{boot_files} - \text{conversation_margin}$. Arithmetic, not chemistry — but the chemical framing reveals that capacity should co-vary with other parameters rather than remaining fixed.

At single-agent scale (~ 75 access events), the framework is under-determined. However, at enterprise scale (thousands of agents, millions of events), three emergent dynamics appear that no existing approach formalizes: phase separation between organizational domains, diffusion of knowledge across agent boundaries, and reaction kinetics of cascading memory updates.

3. Architecture

3.1 Execution Layer

The execution layer follows the blueprint pattern [1]: state machines interleaving deterministic code nodes with agentic subtask nodes.



Deterministic nodes guarantee predictable operations without consuming LLM tokens. Agent nodes handle creative work with scoped context and curated tool access. “Putting LLMs into contained boxes compounds into system-wide reliability” [1].

A centralized MCP server provides the shared capability layer. Adding a tool immediately extends every agent in the fleet. Agents receive curated subsets — “agents perform best when given a smaller box with a tastefully curated set of tools” [1].

Context comes from three sources: - **Rule files** scoped to directories and file patterns, consumed by all agent types - **MCP tool calls** for dynamic context (documentation,

tickets, code intelligence) - **Consolidated memory** from the reconsolidation layer — learned patterns from prior executions

3.2 Observation Layer

Every agent execution generates the three metadata fields that organizational thermodynamics requires: **who** (which agent/team), **when** (timestamps), and **what happened next** (success, failure type, escalation, resolution). No code content or conversation content is required.

Additional telemetry extends the signal:

- **Access patterns:** which knowledge chunks, tools, and rules were accessed, how often, and with what relevance scores
- **Error patterns:** failure type, location, clustering across runs
- **Resolution patterns:** which interventions resolved which failures, in how many iterations
- **Tool usage patterns:** MCP tool call sequences, whether results were used
- **Cadence signatures:** execution frequency distributions and phase relationships

The observation layer applies the reconsolidation principle [4,5]: raw telemetry enters as liquid memory. Access patterns reveal which signals matter. Frequently-accessed patterns consolidate into structured knowledge. Patterns never accessed remain liquid and eventually decay.

Chunk energy is computed per access event: $\alpha_i = \sum \text{sim}(q, \text{chunk}_i) \cdot \exp(-\lambda \cdot \text{age})$. The energy accumulates across executions, producing a landscape of what the organization actually uses versus what merely exists.

3.3 Diagnostic Layer

The diagnostic layer applies the five organizational thermodynamic metrics [3] to agent execution telemetry:

Flow (Little’s Law): throughput of completed agent tasks / inventory of open tasks. Per team, per codebase region. Flow < 1.0 means tasks accumulate faster than they resolve.

Entropy (Shannon): distribution of agent execution outcomes. Five outcome types map directly from [3]:

Original (comms)	Agent equivalent
Resolved (close/merge)	Task completed, PR merged
Escalated (≥ 2 reassignments)	Task bounced between agents/humans
Expanded (≥ 3 participants)	Required additional context/tools
Looped (back-and-forth)	Agent retry cycles (lint $\rightarrow$ fix $\rightarrow$ lint)
Stalled (no activity >7 days)	Task abandoned or blocked

$H = 0$ means all outcomes are identical (perfectly predictable). $H = 1$ means uniformly distributed (maximally unpredictable). High entropy + low flow = swamp.

Cadence (phase correlation): when Team A’s agents start failing, does Team B follow? Correlated cadences reveal hidden dependencies. Anti-correlated cadences — analogous to the $r = -0.96$ holiday decorrelation [2] — reveal structural relationships: one team’s burst follows another’s pause.

Downstream ratio: what fraction of agent output feeds into dependent systems? Low ratio = isolated work. High ratio = critical path.

Fan-out: how many distinct modules does an agent touch per task? Increasing fan-out signals coordination tax or architectural smell.

Classification follows the quadrant model:

	Low Entropy	High Entropy
High Flow	River — healthy	Rapids — productive but fragile
Low Flow	Lake — stable but stagnant	Swamp — friction, needs intervention

Phase transitions between states are detectable through cadence analysis. A river becoming rapids (entropy increasing while flow holds) is an early warning. The diagnostic agent [3] then investigates only the flagged zones — automated attention applied to the agent fleet itself.

3.4 Prescription Layer

The prescription layer matches diagnoses to interventions using the correlation network, calibrated against outcomes over time.

Diagnosis classification:

Pattern	Diagnosis	Signal
Agents lack rules, repeat same mistakes	Knowledge gap	Error clustering without rule file access
API surface too complex for one-shot	Structural problem	High fan-out + high entropy in region
Waiting on another team’s output	Dependency bottleneck	Correlated cadence, low downstream ratio
MCP tools exist but agents can’t find them	Discovery gap	Repeated queries with low relevance scores
No tool exists for a repeated need	Capability gap	Humans creating workarounds (scripts, spreadsheets)

Intervention types, ordered by autonomy:

Level 0 — Visibility: Surface diagnosis to dashboard. “Module X: entropy 3.2σ above baseline.” No automated action.

Level 1 — Knowledge: Auto-generate scoped rule files from error patterns. A cluster of timezone-related failures across migration agents produces a draft rule: “Always normalize to UTC before writing to inventory tables.” Submitted as PR for human review.

Level 2 — Tooling: Auto-generate MCP tools from repeated manual patterns. Three teams independently querying the same data through different methods → draft a shared Toolshed tool. Adding it extends the entire fleet immediately.

Level 3 — Structural: Generate refactoring blueprints from friction analysis. High entropy + high fan-out in a module → draft blueprint to decompose it, with scope estimated from similar past refactors. Human-approved before execution.

Level 4 — Generative: Conceive and build tools from behavioral observation. This extends beyond the agent fleet to the broader organization: support ticket clustering, internal tool drop-off patterns, and workaround detection identify *what people need* before they request it. The system designs a solution, generates a blueprint, executes it through the agent fleet, and ships it for human review.

Each intervention includes a predicted impact (expected entropy reduction, flow improvement) derived from similar past interventions. These predictions are calibrated against measured outcomes through the chemical kinetics framework [6] — interventions that don’t catalyze improvement have their “activation energy” raised, making them less likely to be prescribed in similar contexts.

3.5 Memory Architecture

The system’s memory operates at three timescales, following the consolidation model [4,5]:

Working memory (per-execution): the agent’s context window during a single run. Populated by blueprint context, rule files, and MCP tool results. Lost at execution end.

Consolidating memory (cross-execution): access patterns, error logs, and resolution histories in the observation layer. Raw and liquid. Shaped by the nightly consolidation pipeline [5]: extract access events → compute chunk energy → generate mirror → act on findings during heartbeats. The hot swamp detector identifies content that is frequently accessed but poorly structured — the phase transition point where liquid memory is ready to solidify.

Consolidated memory (organizational): patterns that have proven durable through repeated access across diverse contexts. Scoped rule files. Toolshed tools. Architectural decisions. Diagnostic-intervention correlations.

Promotion follows the solubility product model [6]: a pattern promotes when its access energy exceeds $K_{sp}(\text{semantic_distance}, \text{existing_saturation})$. Novel patterns (high semantic distance from existing consolidated knowledge) promote at lower thresholds because the organizational memory is “unsaturated” in that region.

Decay follows the modified Arrhenius model [6]: $k_{\text{decay}} = A \cdot e^{(-E_a/R)}$ where E_a = co-access depth. Patterns that are part of a web of related accesses (high activation energy) decay slower than isolated observations, regardless of age.

Raw execution transcripts are explicitly NOT indexed into the consolidated layer. Empirical testing [5] demonstrated that unstructured transcripts create semantic traps at scale — a 67:1 volume ratio caused curated content to be completely displaced from search results. Instead, the reconsolidation pipeline extracts structured patterns from execution telemetry and promotes only those patterns that prove their value through repeated, relevant access.

3.6 Cognitive Matching

The cognitive signatures framework [7] extends the prescription layer by matching interventions to the humans best suited to review, own, or collaborate on them.

Five continuous dimensions — Connection Pattern, Frame Dependence, Scope Instinct, Ambiguity Response, Integration Speed — are measured behaviorally, not self-reported. The same behavioral observation that powers the observation layer (who does what, when, and what happens next) feeds cognitive signature estimation at the individual level.

Applications within the architecture:

- **Route reviews:** structural refactoring proposals to high scope-range individuals. Edge case analysis to high precision-seekers.
- **Form teams:** pair explorers (high connection velocity) with refiners (high ambiguity tolerance) for novel problem spaces.
- **Detect capability gaps:** if no one in the organization has the cognitive profile suited to a class of problem, surface as a hiring signal.
- **Match tools to people:** Level 4 interventions (generative) target users whose behavioral patterns indicate they would benefit, rather than broadcasting to everyone.

4. The Flywheel

The architecture produces compounding returns because each cycle enriches subsequent cycles:

More executions → better observation. Larger sample sizes produce more reliable thermodynamic measurements. The chemical kinetics framework [6] becomes better-determined — at ~75 access events it's under-determined; at enterprise scale (thousands of agents, millions of events), the derived parameters outperform hand-tuned constants and reveal emergent dynamics (phase separation between domains, knowledge diffusion across agent boundaries, cascading update kinetics).

Better observation → better diagnosis. Phase transitions are detected earlier. Cadence resonance [2] across teams reveals hidden dependencies — the organizational equivalent of the $r = -0.96$ holiday decorrelation, where anti-correlated agent failure patterns reveal structural coupling invisible to individual teams.

Better diagnosis → better prescription. The correlation network accumulates intervention-outcome pairs. Prescription accuracy improves. The Gibbs free energy model [6] governs whether an intervention is favorable: $\Delta G = \Delta H - T\Delta S$, where ΔH = implementation cost and $T\Delta S$ = organizational entropy $\times$ structural improvement. Interventions with $\Delta G < 0$ proceed; others are deprioritized.

Better prescription → better execution. Agents start with accumulated knowledge (consolidated rule files), better tools (auto-generated MCP tools), and cleaner codebases (structural improvements from prior cycles). One-shot success rates increase — Stripe’s metric for minion effectiveness [1].

Better execution → more capacity. As agents succeed more often with fewer retries, human review time decreases. Freed capacity goes to higher-leverage work — which generates new patterns for the observation layer.

The flywheel is self-correcting. Failed interventions receive negative calibration: their activation energy [6] increases, making them less likely to be prescribed in similar contexts. The system learns what doesn’t work as reliably as what does.

5. Signal Sources Beyond Agents

While agent execution telemetry is the primary signal source, the observation layer accommodates any source that produces the required metadata (who, when, what happened next):

- **Support tickets:** clustering by topic, resolution time, repeat frequency
- **Internal tool usage:** drop-off points in workflows, workaround detection
- **Communication metadata:** message volume, response latency, thread depth [3]
- **Deployment patterns:** rollback frequency, hotfix cadence, change failure rate

Each source feeds the same pipeline: raw events → access pattern tracking → reconsolidation → thermodynamic classification. The diagnostic model operates identically regardless of source because it measures metadata properties, not content.

The generative prescription level (Level 4) becomes most powerful when multiple signal sources converge: agent failures in a codebase region + support tickets from users of that feature + tool usage dropping off at a specific workflow step = high-confidence diagnosis with a clear intervention.

6. Safety and Human Oversight

The autonomy levels provide a graduated trust model:

- **Levels 0-1** (visibility and knowledge) produce information and documentation, not code changes. Safe by default.
- **Level 2** (tooling) generates MCP tools that require human review before deployment to the fleet.
- **Level 3** (structural) generates blueprints requiring human approval before execution and review of results.

- **Level 4** (generative) requires human approval at conception (should we build this?), design (is this the right solution?), and deployment (does it work?).

The system never escalates its own autonomy level. Organizations configure permitted levels per zone. Agent execution occurs in isolated environments with no access to production data or external networks — the same safety property Stripe leverages for minions [1].

All interventions log full provenance: which observations → which diagnosis → which prescription → which outcome. The audit trail enables human review of the system’s reasoning at any point, following the automated attention principle [3] — the system identifies where to look; humans decide what to do.

7. Discussion

7.1 Relationship to existing systems

CI/CD dashboards show what happened but don’t diagnose why. Project management tools track human-assigned priorities rather than measured friction. AI coding assistants optimize individual developer productivity rather than organizational capability. This architecture operates at a different level: it measures organizational health from execution patterns and intervenes structurally.

7.2 The complexity ladder realized

Each paper contributed one rung:

Layer	Discipline	Contribution	Applied here as
Cadence [2]	Mathematics	Counting + time = signal	Sensing primitive for all telemetry
Org Thermo [3]	Physics	Entropy, flow, phase states	Diagnostic framework
Chemical Kinetics [6]	Chemistry	Saturation, catalysis, decay	Memory parameter derivation
Dreaming [5]	Biology	Sleep, consolidation, adaptation	Infrastructure-driven improvement
Cognitive Signatures [7]	Psychology	Identity, attention, collaboration	Human-AI matching

The claim that “natural language metaphors encode real physics” [2,3,6] is tested by this architecture. When an engineer says a codebase region is “swampy,” the system can compute whether this is literally true (low flow, high entropy) and prescribe a specific intervention with a predicted outcome.

7.3 Why now

Three prerequisites converged: (1) autonomous coding agents reliable enough for unattended operation at scale [1], (2) MCP as a standard protocol for agent-tool integration, and (3) embedding models cheap enough to run locally for persistent memory without API dependencies. The individual components existed; the connective tissue to close the loop was missing.

7.4 Limitations

The architecture requires sufficient agent execution volume for statistically meaningful signals — organizations with fewer than ~100 agent runs per week may not generate enough telemetry. The chemical kinetics parameters [6] need ~500+ events before derived values outperform hand-tuned constants. Cognitive signature estimation [7] requires behavioral observation data that accumulates slowly. And the correlation network needs months of intervention-outcome pairs before prescription accuracy exceeds naive baselines.

8. Conclusion

We present an architecture that transforms agent fleets from stateless executors into an organizational learning system. By applying thermodynamic measures to agent execution telemetry, consolidating patterns through access-weighted memory, and feeding learned interventions back into the agent fleet, the system creates a self-improving flywheel where every agent run makes the next one smarter.

The key contributions are: (1) connecting organizational thermodynamics [3] to agent execution telemetry as a novel signal source requiring no code content access, (2) demonstrating empirically [5] that raw transcript indexing creates semantic traps while access-driven promotion preserves retrieval quality, (3) applying chemical kinetics [6] to derive memory and intervention parameters that co-vary with system state rather than remaining fixed, (4) composing cadence resonance [2] as the sensing layer that makes cross-team pattern detection possible without predictive models, and (5) closing the loop from observation through diagnosis and prescription back to execution, creating compounding returns.

Each component delivers standalone value at current scale. Together, they describe an organization that gets smarter every time an agent runs.

References

- [1] A. Gray, “Minions: Stripe’s one-shot, end-to-end coding agents,” Stripe Engineering Blog, Parts 1 & 2, February 2026.
- [2] E. Gill and K. Ash, “Cadence Resonance: Counting and Time as Universal Signal Primitives,” 2026. DOI: 10.5281/zenodo.18808102.
- [3] E. Gill and K. Ash, “Organizational Thermodynamics: Automated Attention from Communication Metadata,” 2026. DOI: 10.5281/zenodo.18808102.

- [4] E. Gill and K. Ash, "Embedding Trajectory Compression for Persistent Agent Memory: SVD, DCT, and Access-Driven Reconsolidation," 2026. DOI: 10.5281/zenodo.18808102.
- [5] E. Gill and K. Ash, "Dreaming in Access Patterns: A Self-Improving Memory Architecture for Persistent AI Agents," 2026. DOI: 10.5281/zenodo.18808102.
- [6] E. Gill and K. Ash, "Chemical Kinetics as a Framework for Multi-Agent Memory Management," 2026. DOI: 10.5281/zenodo.18808102.
- [7] E. Gill and K. Ash, "Cognitive Signatures: Measuring Cross-Domain Collaboration Potential," 2026.